

# 杭州电子科技大学

## 实验报告

课程名称：密码学基础与算法课程设计 姓名:xx 学号: xx

实验地点：4 教 441-443-445-447

实验时间：周五 3-5 节

### 一、实验名称：RSA 密码实验

### 二、实验要求：

- 1、了解公钥密码的起源与涵义。
- 2、掌握 RSA 密码的加解密原理。
- 3、用 Visual C++实现 RSA 密码并输出结果。

### 三、实验内容：

1、公钥密码体制的概念由 Diffie 和 Hellman 于 1976 年提出，用于解决对称密码体制中密钥分配的问题。在公钥密码体制中，密钥被分为公钥与私钥，公钥是公开的，用于加密；私钥是保密的，用于解密。经过四十余年的研究发展，RSA 密码、ElGamal 密码、椭圆曲线密码等等公钥密码体制在商业、军事上都已经得到了广泛的应用。

2、RSA 密码由 R. Rivest、A. Shamir 和 L. Adleman 于 1977 年提出，是最著名的公钥密码，能够抵抗到目前为止已知的绝大多数密码攻击，已被 ISO 推荐为公钥数据加密标准。RSA 密码的安全性基于这样一个事实：将两个大素数  $p, q$  相乘十分容易，但对其乘积  $N=pq$  作因子分解却极其困难。在本实验中，假设  $p, q$  均为  $len$  比特素数，具体描述如下：

#### 系统参数

大素数  $p, q$ ：  $len$  比特素数（即  $2^{len-1} \leq p, q < 2^{len}$  且  $p, q$  为素数），保密；

乘积  $N$ ：  $N = p * q$ ，约为  $2 * len$  比特，公开；

$N$  的欧拉函数值  $\varphi(N)$ ：  $\varphi(N) = (p-1)(q-1)$ ，保密。

#### 公钥

$e$ ：满足  $1 < e < \varphi(N)$  且  $\gcd(e, \varphi(N)) = 1$

## 私钥

$d$ : 满足  $1 < d < \varphi(N)$  且  $ed \equiv 1 \pmod{\varphi(N)}$ , 即  $d = e^{-1} \pmod{\varphi(N)}$

## 明文

正整数  $m$ : 满足  $1 < m < N$

## 加密

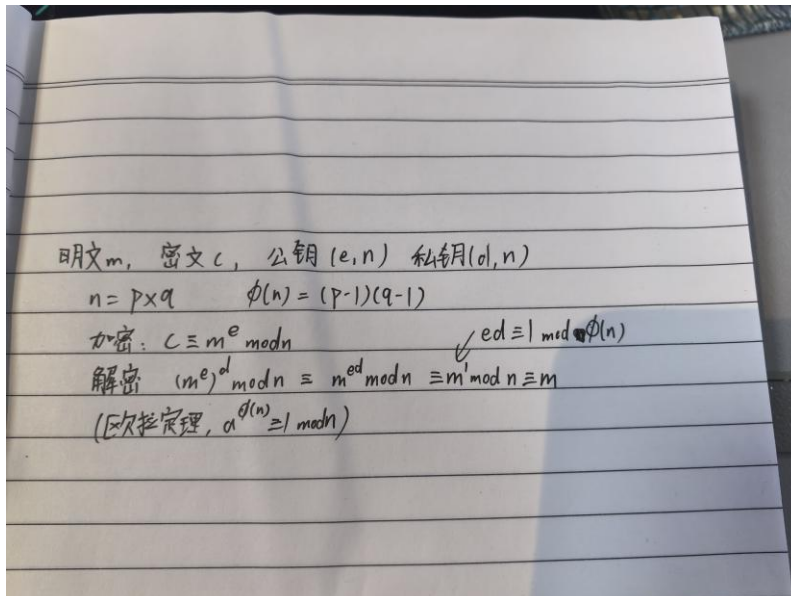
密文  $c = m^e \pmod{N}$

## 解密

密文  $m = c^d \pmod{N}$

## 思考题:

为什么 RSA 密码能正确解密? 请在此写出详细的证明过程。



3、使用 Visual C++ 编写程序, 实现 RSA 密码并输出结果。

编程的关键是实现大整数运算, 包括模加、模乘、模幂等等运算, 所以考虑使用无符号字符数组表示大整数, 即使用 **256 进制** 表示大整数, 用结构体实现。

- 1) 新建一个空项目, 取名 `rsa`。
- 2) 添加 `cpp` 文件, 取名为 `rsa.cpp`。
- 3) 在 `rsa.cpp` 文件中先写入

```

#include<time.h>
#include<iostream>
using namespace std;

#define SIZE 33          //SIZE 是数组长度, 能表示的最大整数的 bit 数是 8*SIZE,
                        //SIZE 可自由选取, 本实验默认取为 33

typedef struct Bigint    //用于表示大整数的无符号字符数组
{
    unsigned char num[SIZE];
}Bigint;

typedef struct Bigint2   //大整数乘法可能需要的数组长度是原数组的两倍
{
    unsigned char num[2*SIZE];
}Bigint2;

```

#### 4) 可能需要的函数

```

//初始化
Bigint Init(unsigned char a[], int length);
//复制
void Copy(Bigint &a, Bigint b);
//打印输出
void Print(Bigint a);
//计算数组长度
int Length(Bigint a);
int Length(Bigint2 a);
//比较大小
int Compare(Bigint a, Bigint b); //a>b, a=b, a<b 分别输出 1, 0, -1
int Compare(Bigint2 a, Bigint2 b); //a>b, a=b, a<b 分别输出 1, 0, -1
//左移 loop 个字节
Bigint MoveLeft(Bigint a, int loop);
Bigint2 MoveLeft(Bigint2 a, int loop);
//右移一个比特
void BitMoveRight(Bigint &a);
//扩充数组
Bigint2 Extend(Bigint a);
//截断数组
Bigint Narrow(Bigint2 a);
//生成随机数
Bigint BigRand(Bigint n); //生成 1 到 n 之间的随机数
Bigint BigRandOdd(int bytes); //生成 bit 数是 8*bytes 的随机奇数

//基本运算
Bigint Add(Bigint a, Bigint b); // 加法: 输入 a, b, 返回 a + b

```

```

Bigint Sub(Bigint a, Bigint b); // 减法: 输入 a>b, 返回 a - b
Bigint2 Sub(Bigint2 a, Bigint2 b); // 减法: 输入 a>b, 返回 a - b
Bigint2 Mul(Bigint a, Bigint b); // 乘法: 输入 a,b, 返回 a * b
Bigint Div(Bigint a, Bigint b); // 除法: 输入 a,b, 返回 a / b
Bigint Mod(Bigint a, Bigint b); // 求余: 输入 a,b, 返回 a % b
Bigint2 Mod(Bigint2 a, Bigint2 b); // 求余: 输入 a,b, 返回 a % b

Bigint AddMod(Bigint a, Bigint b, Bigint n); // 模加: 计算 a + b mod n
Bigint SubMod(Bigint a, Bigint b, Bigint n); // 模减: 计算 a - b mod n(要求 a>b)
Bigint Sub2Mod(Bigint a, Bigint b, Bigint n); // 模减: 计算 a - b mod n
Bigint MulMod(Bigint a, Bigint b, Bigint n); // 模乘: 计算 a * b mod n
Bigint PowMod(Bigint a, Bigint b, Bigint n); // 模幂: 计算 a ^ b mod n

//MillerRabin 素性检测
bool MillerRabinKnl(Bigint &n); //单次素性检测, 通过返回 1, 否则返回 0
bool MillerRabin(Bigint &n, long loop); //loop 次素性检测, 通过返回 1, 否则返回 0

//生成 bit 数是 8*bytes 的素数
Bigint GenPrime(int bytes);

//生成私钥
bool Inverse(Bigint e, Bigint phiN, Bigint &d); //计算  $d = e^{-1} \bmod \varphi(N)$ 
//加密
Bigint Encrypt(Bigint m, Bigint e, Bigint N); //计算  $c = m^e \bmod N$ 
//解密
Bigint Decrypt(Bigint c, Bigint d, Bigint N); //计算  $m = c^d \bmod N$ 

```

## 5) 编写主函数, 主要步骤有

生成并输出大素数 p、q 及其乘积  $N=pq$ ;

输出公钥 e;

输出私钥 d;

随机生成并输出明文;

加密并输出密文;

解密并输出解密后的明文。

如下所示:

```
D:\Program Files (x86)\Microsoft Visual Studio 10.0\VC\vcprojects\RSA\Debug\RSA.exe
生成大素数:
p=279405879024828246713180947787608738547
q=240845450798658244373270493430610093081

系统参数是:
N=67293634889530129061886816553891988938806440043090984832964793595132662693307

公钥是:
e=65537

私钥是:
d=52506634629461413853375104922407506391018852648757904621973708048622442304513

明文是:
m=31628633767000880302747882090455429665585021074826914259669911205492665142248

加密得到的密文是:
c=60340565441035184476991802312618884264757871618352537790312329833011683150744

解密得到的明文是:
31628633767000880302747882090455429665585021074826914259669911205492665142248

请按任意键继续. . .
```

**提示点:**

- 1) 无符号字符的范围是 0~255，所以大整数是用 256 进制表示的。  
假设数组 a 里元素分别为 a.num[0], a.num[1],……,a.num[SIZE-1]  
则 a 表示的数为  
$$a.num[0] * 1 + a.num[1] * 256 + a.num[SIZE-1] * 256^{SIZE-1}$$
- 2) 为了加密速度快，本次实验的公钥 e 可选定 65537(实际当中也很常用)，  
其 256 进制表示是{1,0,1}。
- 3) 实验中涉及到的数都是非负数，所以减法运算需要先判断大小。
- 4) 大整数乘法已用 2 倍长数组保证不溢出，而大整数加法可能会溢出，所以  
SIZE 定义时要比 N 的实际大小多一点。比如 N 是 80bit(10 字节)，那么  
SIZE 应该定义成 11。
- 4) 测试小整数示例时可用 Bigint a = {0}, Bigint a = {1}, Bigint a={15}类似的  
初始化命令。
- 5) 部分函数参考代码在”RSA 密码\_部分参考代码.doc”中有提供

**四、实验报告:**

本次实验要求实现 p, q 均为 128bit(16 字节)的情形，SIZE 可定义为 33。

**1) RSA 密码程序代码如下:**

```
import random

def extended_gcd(a, b):
    # 扩展欧几里得算法，返回 (gcd, x, y)，使得  $a * x + b * y = \text{gcd}$ 
    if b == 0:
        return a, 1, 0
    gcd, x1, y1 = extended_gcd(b, a % b)
    x = y1
    y = x1 - (a // b) * y1
    return gcd, x, y

def mod_inverse(a, m):
    # 求模反元素
    gcd, x, y = extended_gcd(a, m)

    # 如果  $\text{gcd}(a, m) \neq 1$ ，则不存在模反元素
    if gcd != 1:
        raise ValueError(f'{a} and {m} are not coprime, so no modular inverse exists.')

    # x 是 a 的模反元素，但 x 可能是负数，我们需要确保返回的 x 是正数
    return x % m

# 生成一个指定字节数的随机大整数
def get_random_number(bits):
    return random.getrandbits(bits)
```

# 快速模幂运算

```
def mod_exp(base, exp, mod):  
    return pow(base, exp, mod)
```

# 判断一个数是否是素数

```
def miller_rabin(n, k=5):  
    # 特殊情况处理：小于 2 的数不是素数，2 和 3 是素数  
    if n <= 1:  
        return False  
    if n <= 3:  
        return True  
    if n % 2 == 0 or n % 3 == 0:  
        return False
```

# 找到  $r$  和  $s$ ，使得  $n - 1 = 2^r * s$ ，其中  $s$  是奇数

$s = n - 1$

$r = 0$

while  $s \% 2 == 0$ :

$s //= 2$

$r += 1$

# 进行  $k$  次检验

```
for _ in range(k):  
    a = random.randint(2, n - 2)  
    x = mod_exp(a, s, n) # 计算  $a^s \bmod n$   
    if x == 1 or x == n - 1:  
        continue  
    for _ in range(r - 1):  
        x = mod_exp(x, 2, n)  
        if x == n - 1:
```

```
                break
            else:
                return False
        return True

# 生成大素数
def generate_prime(bits):
    while True:
        num = get_random_number(bits)
        if miller_rabin(num):
            return num

# 计算欧拉函数  $\phi(N)$ 
def euler_totient(p, q):
    return (p - 1) * (q - 1)

# 生成公钥和私钥
def generate_keys(bits=128):
    # 生成两个大素数 p 和 q
    p = generate_prime(bits)
    q = generate_prime(bits)

    # 输出生成的大素数 p 和 q
    print(f"生成的大素数 p: {p}")
    print(f"生成的大素数 q: {q}")

    # 计算  $N = p * q$ 
    n = p * q
```

```
# 计算欧拉函数  $\phi(N)$ 
phi_n = euler_totient(p, q)

# 选择公钥 e
e = 65537

# 计算私钥 d, d 是 e 的模反元素
d = mod_inverse(e, phi_n)

# 返回公钥 (e, n) 和私钥 (d, n)
return (e, n), (d, n)

# RSA 加密
def encrypt(plain_text, public_key):
    e, n = public_key
    cipher_text = mod_exp(plain_text, e, n)
    return cipher_text

# RSA 解密
def decrypt(cipher_text, private_key):
    d, n = private_key
    plain_text = mod_exp(cipher_text, d, n)
    return plain_text

# 将字符串转为整数
def string_to_int(s):
    return int.from_bytes(s.encode(), 'big')

# 将整数转为字符串
def int_to_string(i):
```

```
        byte_length = (i.bit_length() + 7) // 8
        return i.to_bytes(byte_length, 'big').decode()

# 主函数
def main():
    # 生成公钥和私钥
    public_key, private_key = generate_keys(bits=128)

    # 输出公钥和私钥
    print(f"公钥: (e, n) = ({public_key[0]}, {public_key[1]})")
    print(f"私钥: (d, n) = ({private_key[0]}, {private_key[1]})")

    # 输入明文
    plain_text = input("请输入明文: ")
    plain_int = string_to_int(plain_text)

    # 加密
    cipher_text = encrypt(plain_int, public_key)
    print(f"加密后的密文: {cipher_text}")

    # 解密
    decrypted_text = decrypt(cipher_text, private_key)
    decrypted_string = int_to_string(decrypted_text)
    print(f"解密后的明文: {decrypted_string}")

if __name__ == "__main__":
    main()
```

## 2) RSA 密码输出界面截屏如下：

```
BT (p) = 254853732240377438061523355204319439
BT (q) = 137105910873855295131135201172897684553
BT (e) = 65537
BT (n) = 34941953098418570791080128097143868186359258448111891576230832227267925767
BT (d) = 7143861171028738423023370559739241771648546912265976753211926225813268625
BT (m) = Hi,this is RSA!
BT (c) = 24668788674019108730372693314889634265212447980174528240004560836183189503
BT (m) = Hi,this is RSA!
```